Week 3 - Univariate Analysis

Please run the cells of the notebook as you get to them while reading

In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
```

1. Lesson: Bar charts and univariate graphs

Let's make a dataset (in this case, just a series) which is weighted to have more small values than large values. By squaring a random number between 0 and 1, we ensure that half (those whose initial value is below 0.5) are below 0.25, while the other half are between 0.25 and 1. This means that most of the values are small, and it's more interesting than analyzing a perfectly uniform dataset. This kind of trick - transforming one random variable to get another - can generate a variety of random datasets for you. We then multiply by 100 to get a number between 0 and 100.

In [2]:
```
np.random.seed(0)
lesson_series = np.round(np.random.random(size = 1000)**2 * 100, 2)
lesson_series[0:10] # check the first ten values.  Are they mostly on the small sid
```

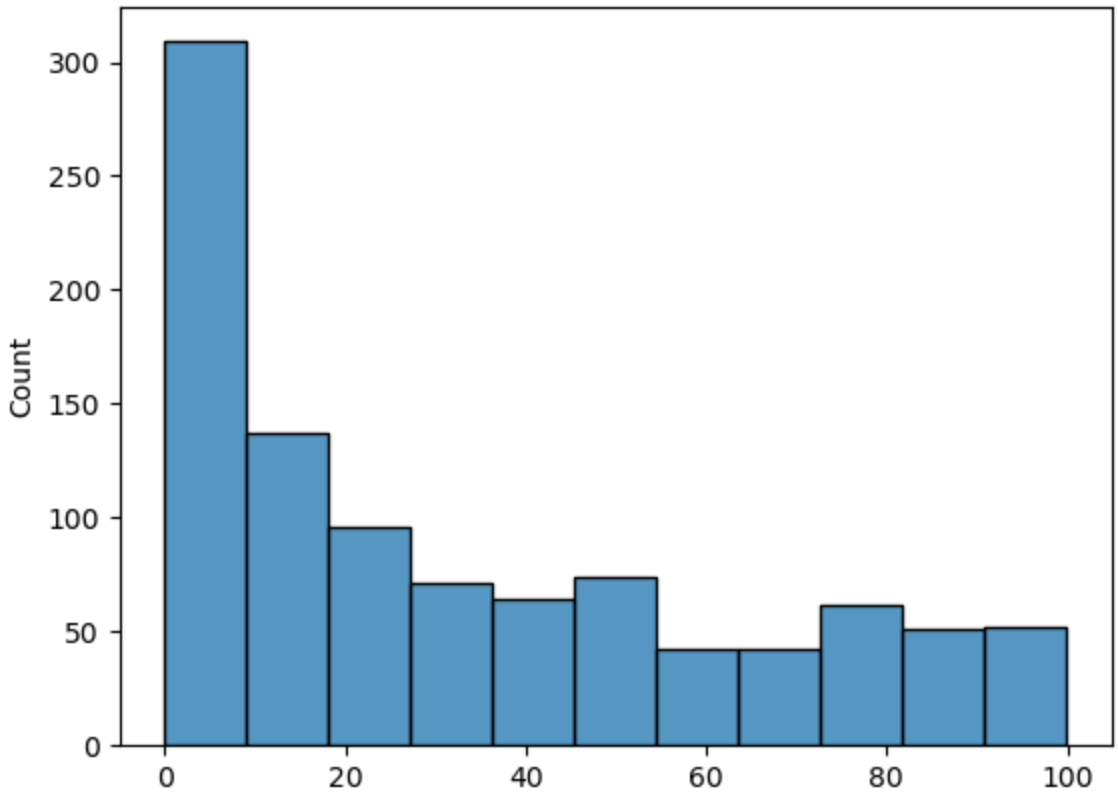
Out[2]:
```
array([30.12, 51.15, 36.33, 29.69, 17.95, 41.72, 19.15, 79.53, 92.86,
       14.7 ])
```

In [3]:
```
import seaborn as sns
```

In the plot below, you can see a histogram of the values in the series. For some reason, it decided to have exactly 11 bins (we allowed it to choose the number of bins.) Most values - about 300 of them - are between 0 and 9, and the next most likely bin is between 9 and 18. Since there are 1000 values, the total of the bars should be 1000.

In [4]:
```
sns.histplot(lesson_series)
```

Out[4]:
```
<Axes: ylabel='Count'>
```



There are 11 bins or bars, a number which by default is chosen by seaborn. We can reproduce this manually to (hopefully) see the same values as numbers. I'm not sure that this second histogram is guaranteed to be exactly the same, but it looks the same to me:

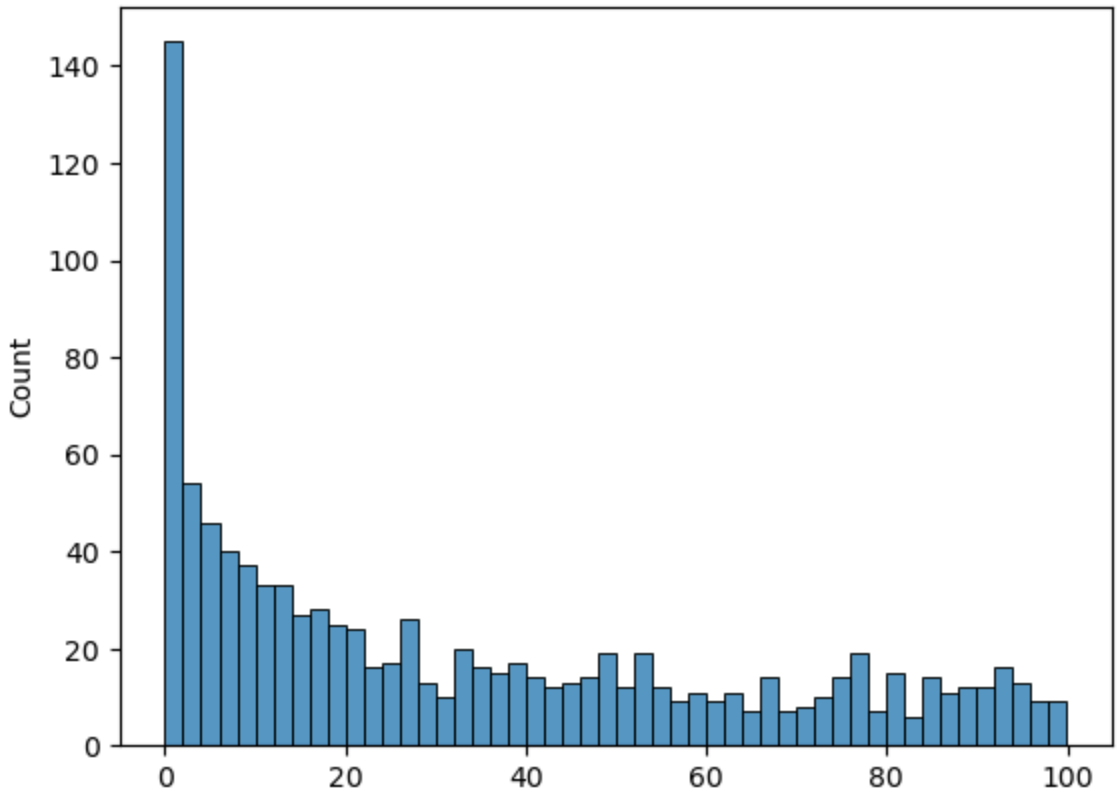
In [5]:
```
np.histogram(lesson_series, bins = 11)[0]
```

Out[5]:
```
array([309, 137,  96,  71,  64,  74,  42,  42,  62,  51,  52])
```

What happens if we override seaborn and choose the number of bins ourselves? We could choose a much larger number of bins:

In [6]:
```
sns.histplot(lesson_series, bins = 50)
```

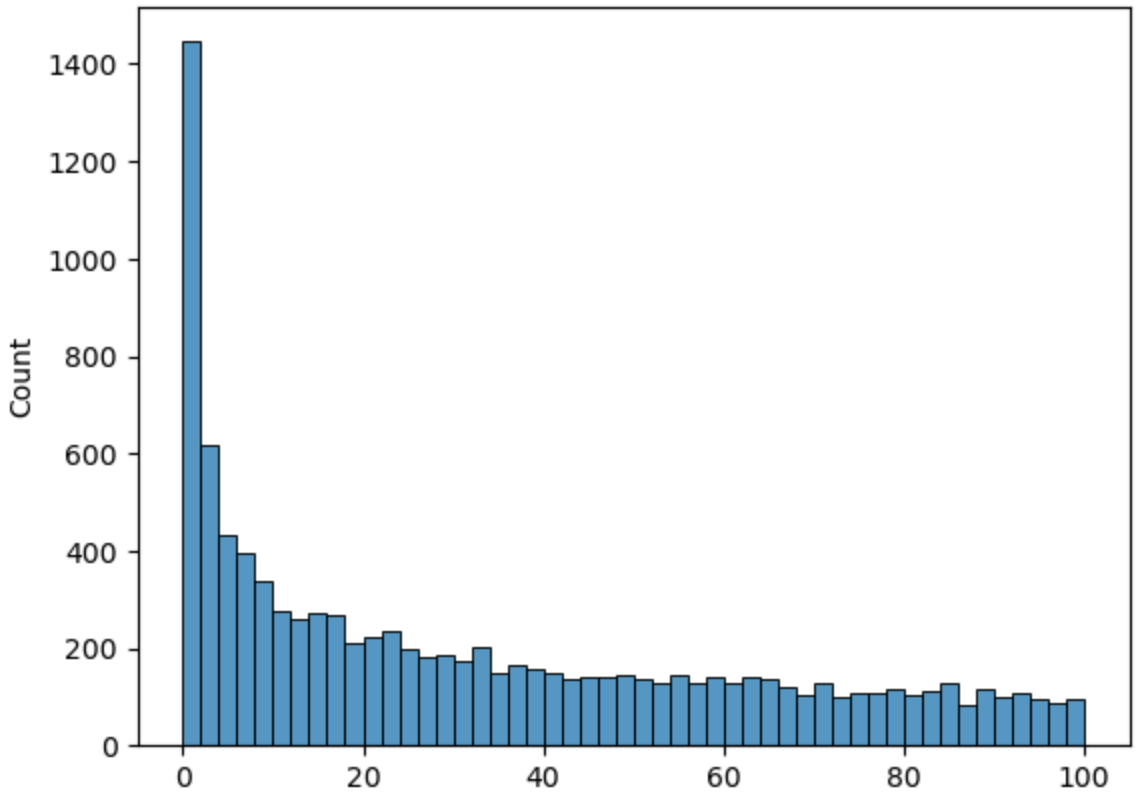
Out[6]:
```
<Axes: ylabel='Count'>
```



One disadvantage of this 50 bin picture is that the outliers are worse. That is, the graph wobbles up and down a bit more randomly. That's because there are fewer values in each bin, so there's more of a role for chance to take effect. If we had many more data points and/or fewer bins, we could get rid of this wobble.

In [7]:
```
lesson_series_2 = np.round(np.random.random(size = 10000)**2 * 100, 2)
sns.histplot(lesson_series_2, bins = 50)
```

Out[7]:
```
<Axes: ylabel='Count'>
```

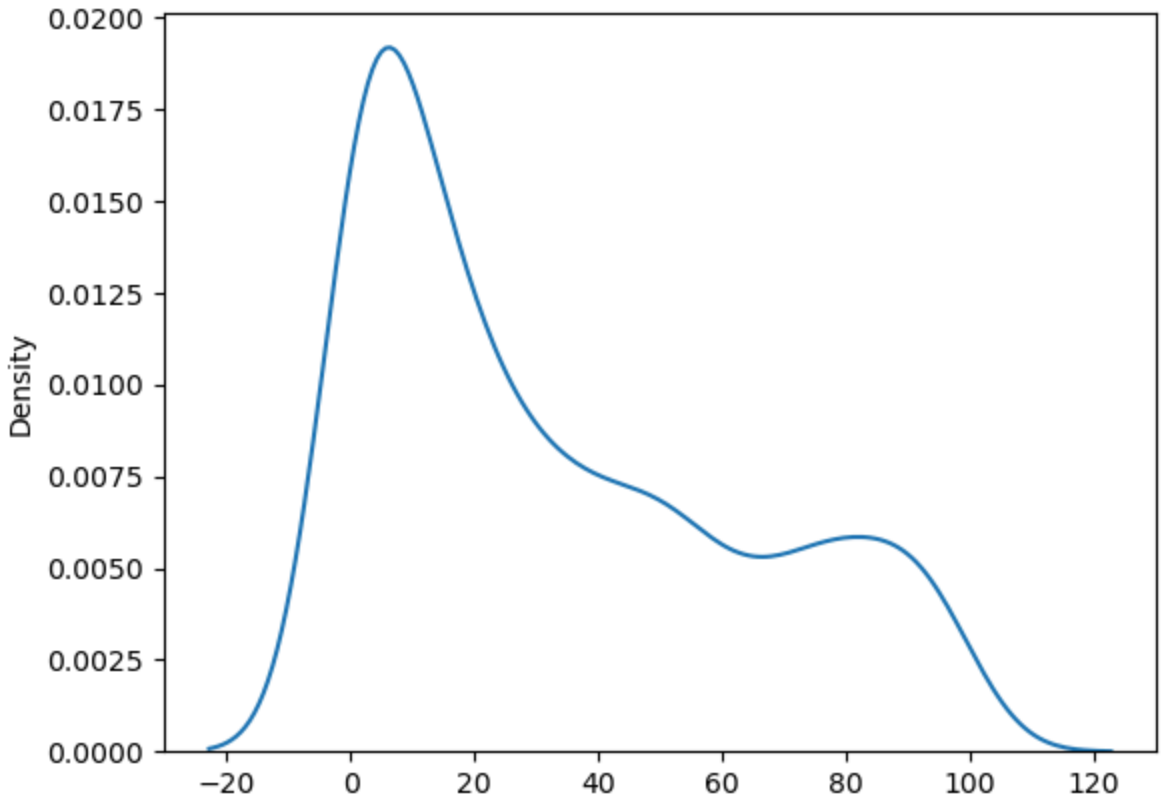


Here you can see that because the number of bins is the same as in the second graph above, but for more data, the histogram is a smoother graph. Why does more data make for a smoother graph? Something for you to think about. I said it's because a larger number of data points in each bin reduces the role of chance - but why is that?

Here is a KDE (Kernel Density Estimate) plot. It's just the same histogram, but drawn smoothly. The KDE plot doesn't have a "number of bins." It's always drawn the same way. In this case, because of the smoothness of the curve, it seems that x-values less then zero and above 100 are still plotted, even though there were no such values in the dataset. This seems like a drawback of the KDE plot, especially if the viewer is unprepared for this aspect of the plot.

In [8]:
```
sns.kdeplot(lesson_series)
```

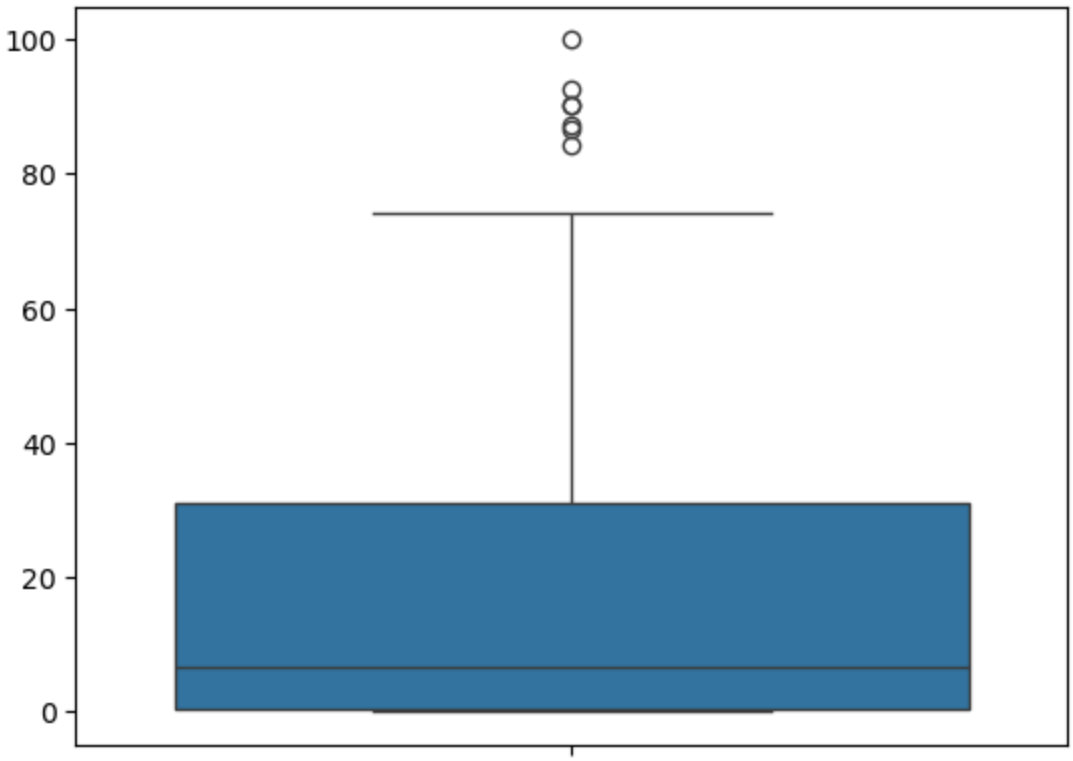
Out[8]:
```
<Axes: ylabel='Density'>
```



We could also draw a box plot. This time, to construct the data I used a fourth power rather than squaring, with only 100 data points, so that the points are even more concentrated toward the small numbers. It turns out that this will create a more interesting boxplot. The top and bottom edges of the box are the 75th and 25th percentile, respectively, and the top and bottom "whiskers" show a larger range which is a multiple of 1.5 times the the box height. (The bottom whisker cannot be see because it's pushed against the bottom of the graph.) The filled-in box shows that half of the values are between about 0 and 30 on the y-axis. Is that what you'd expect? The 25th and 75th percentile of the original uniform random variable are at 0.25 and 0.75. Taken to the fourth power and multiplied by 100 (remember, that's how we constructed our sample), that's 0.25**4 * 100 = 0.4 and 0.75**4 * 100 = 32. It's plausible that those are the height of the bottom and top of the box. We can see that a small number of samples are above the top whisker; they are shown as individual dots.

In [9]:
```
lesson_series_3 = np.round(np.random.random(size = 100)**4 * 100, 2)
sns.boxplot(lesson_series_3)
```

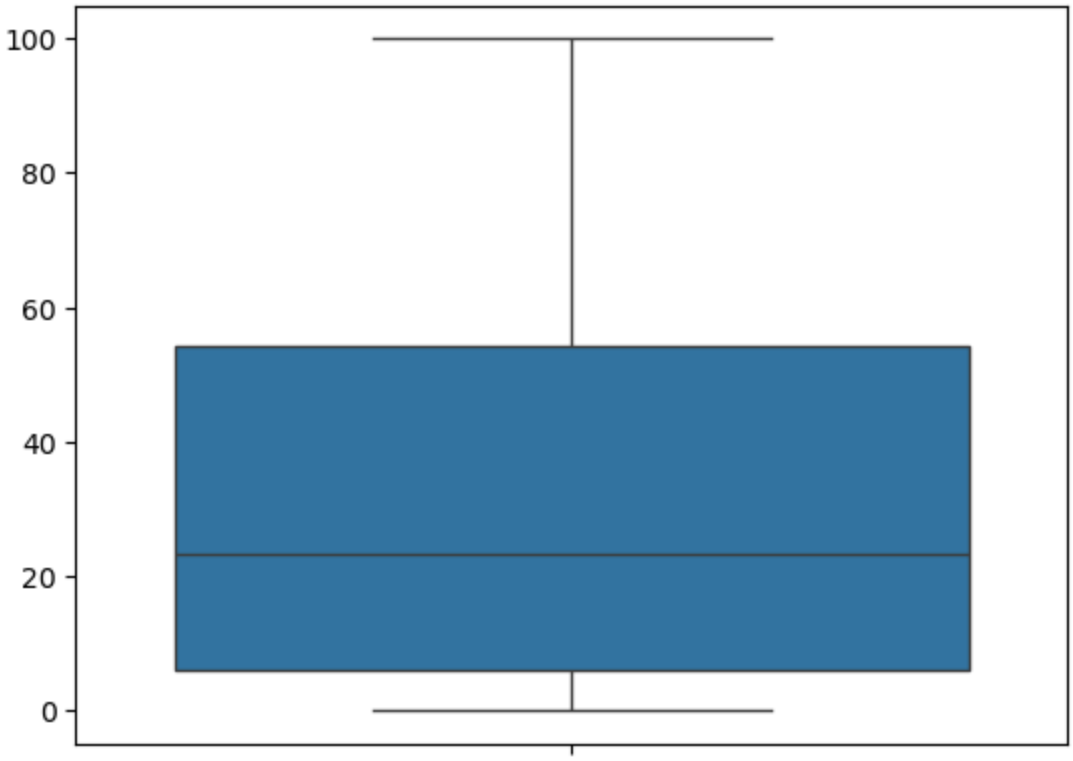
Out[9]: <Axes: >



If we go back to the original lesson_series with the squared values, there will be two whiskers, because it isn't so strongly weighted toward small values:

In [10]:
```
sns.boxplot(lesson_series)
```

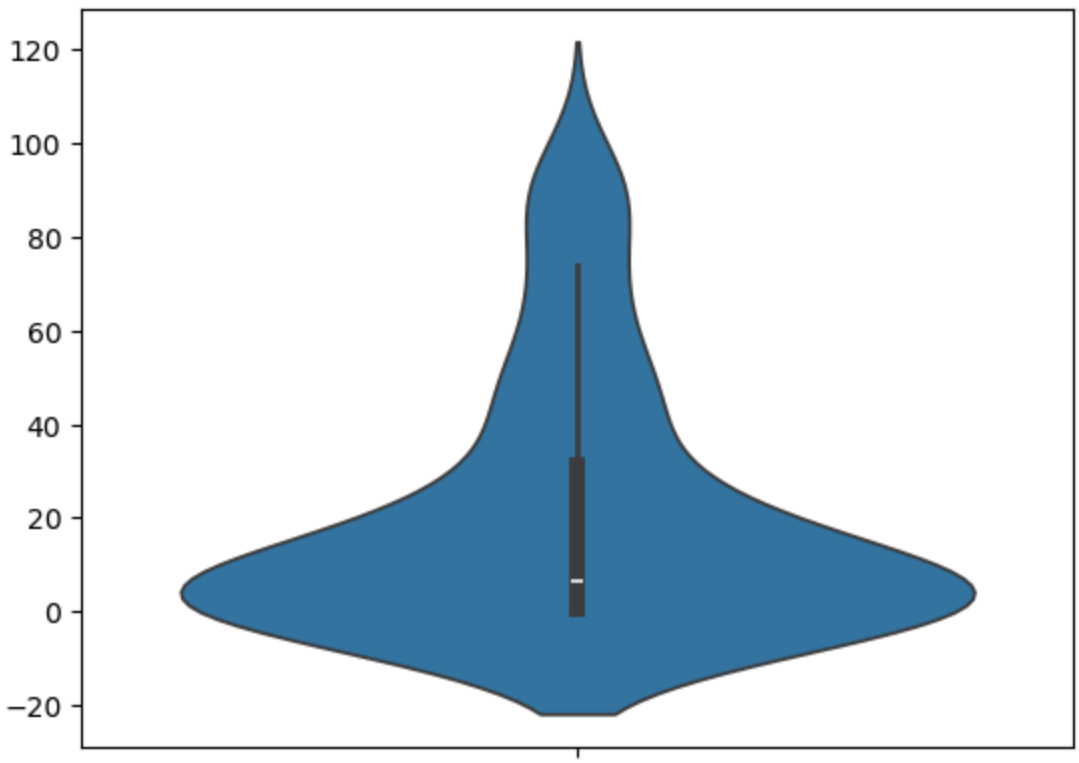
Out[10]: <Axes: >



Going back to the fourth power series, another histogram variant is the violin plot. This simply combines a kde plot (turned on its side and forming two side of the violin) with a boxplot:

In [11]:
```
sns.violinplot(lesson_series_3)
```

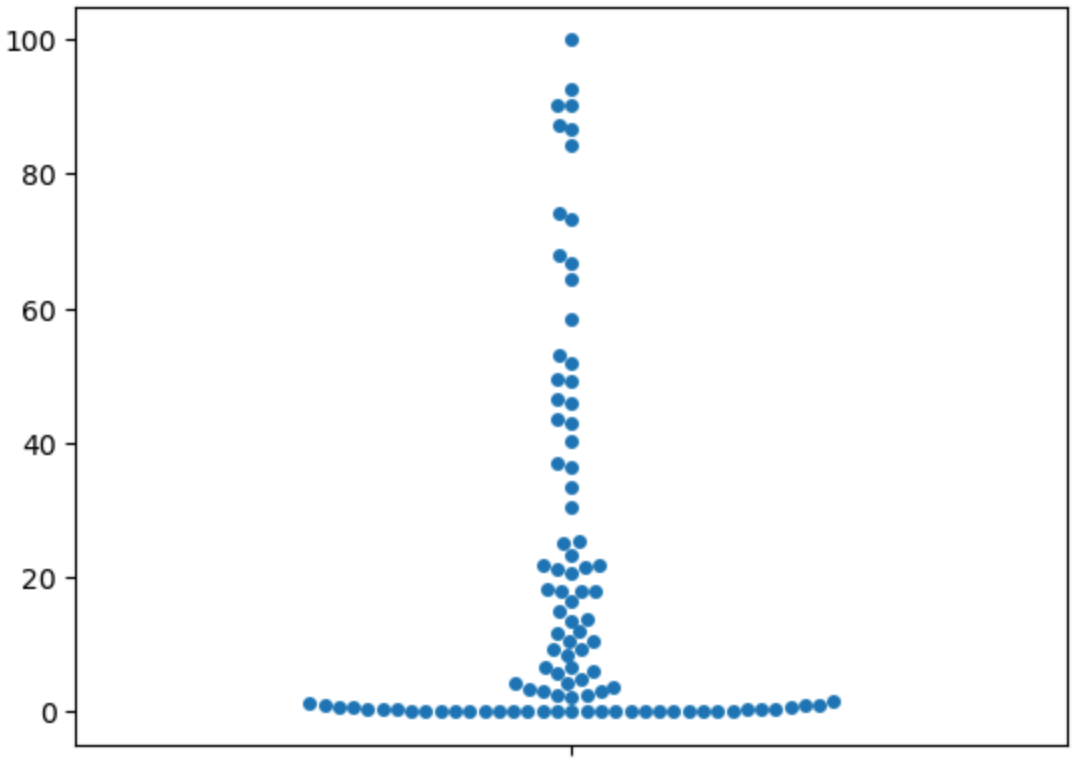
Out[11]:
```
<Axes: >
```



Finally, a swarm plot shows the histogram (turned on its side and doubled, as with the violin plot) but showing each individual point.

In [12]:
```
sns.swarmplot(lesson_series_3)
```

Out[12]:
```
<Axes: >
```



2. Weekly graph question

Below are a histogram and table representation of the same data. A species of bird is being analyzed, and each individual's body length in inches has been measured.

Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

In [13]:
```
import numpy as np
import pandas as pd

np.random.seed(0)
num_data = 10000
data = np.random.normal(size = num_data) + 6
df = pd.DataFrame(data.T, columns = ["data"])
```

In [14]:
```
histnums = np.histogram(df["data"])
histcounts = histnums[0]
histmins = histnums[1][0:-1]
histmaxes = histnums[1][1:]
```

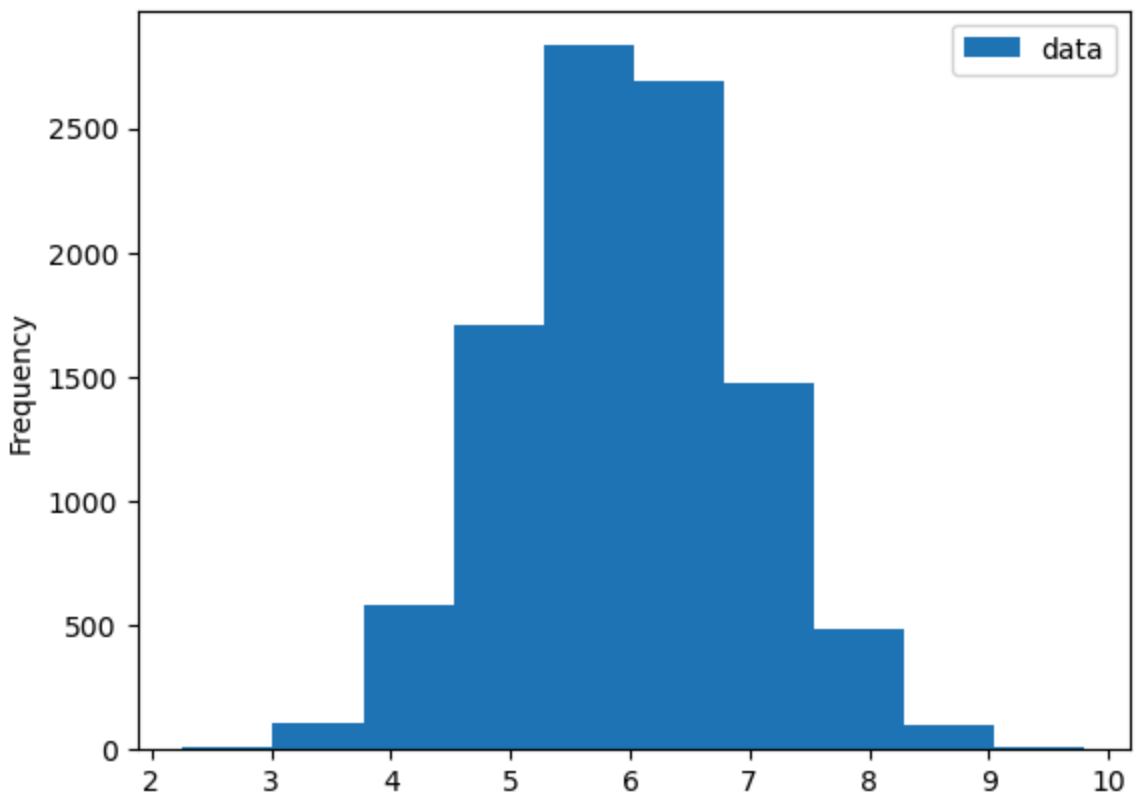
In [15]:
```
pd.DataFrame(np.array([histcounts, histmins, histmaxes]).T, columns = ["count", "mi
```

Out[15]:

	count	minval	maxval
0	10.0	2.259899	3.014075
1	110.0	3.014075	3.768252
2	579.0	3.768252	4.522428
3	1710.0	4.522428	5.276604
4	2833.0	5.276604	6.030780
5	2688.0	6.030780	6.784956
6	1479.0	6.784956	7.539132
7	487.0	7.539132	8.293308
8	97.0	8.293308	9.047484
9	7.0	9.047484	9.801660

In [16]:
```
df.plot.hist()
```

Out[16]: <Axes: ylabel='Frequency'>



Changes to the graph and a comparison of the graph and the table.

1. Histogram
   - Pros
     A. A graph is easier to read and determine what the data means

B. the graph is much better in this case because the table shows data with too many decimal places for one to easily comprehend/understand.
C. The graph shows a true representation of the data visually

- Cons
  A. The histogram's bars are too close together to make it easily readable
  B. The histogram's legend does not help
  C. The histogram does not show values
- Graph Suggestions
  A. The histogram would be more informative if it had axis labels and data labels to better inform the consumer of the graph as to what is being represented.
  B. The distance between the histogram's bars could be increased
  C. In this case the graph legend is fairly useless. It should either be eliminated or should describe the data instead of just stating that it is data

2. Table
   - Pros
     A. The data is represented in such a way that it is more exact (more decimal places)
     B. There is nothing to distract one from the actual data
   - Cons
     A. There is too much to take in with all the decimal places
     B. It is relatively unclear what is being represented here. Some context in the table would be helpful (this is true of both the histogram and the table)
   - Suggestions
     A. Column labels that are more descriptive

3. Homework - Amusement Park Rides

Now let's imagine we have some data about how many times different visitors to an amusement park used each ride, as well as how much money they spend at the amusement park. Each sample represents a single visit by a single visitor on a given date.

In [17]:
```
num_visits = 10000
np.random.seed(0)
```

In [18]:
```
df = pd.DataFrame(columns = ["VisitDate"])
```

In [19]:
```
start = datetime(2010, 1, 1)
end = datetime(2024, 1, 1)
numdays = (end - start).days
random_days = np.random.randint(0, numdays, size = num_visits)
s = start + pd.to_timedelta(random_days, unit='D')
s = s.sort_values()
df["VisitDate"] = s
```

In [20]:
```
df["IsAdult"] = np.random.choice([True, True, False], size = num_visits)
```

In [21]:
```
df["MartianRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 3 + [3] * 1 + [10],
```

In [22]:
```
df["TeacupRide"] = np.random.choice([0] * 2 + [1] * 5 + [2] * 3 + [5] * 2, size = n
```

In [23]:
```
df["RiverRide"] = np.random.choice([0] * 8 + [1] * 3 + [2] * 2, size = num_visits)
```

In [24]:
```
df["MoneySpent"] = np.round(np.random.random(size = num_visits)**2 * 100, 2)
```

In [25]:
```
df.iloc[0:5]
```

Out[25]:

	VisitDate	IsAdult	MartianRide	TeacupRide	RiverRide	MoneySpent
0	2010-01-01	False	0	1	4	10.30
1	2010-01-01	True	3	0	0	38.77
2	2010-01-01	True	0	0	0	79.34
3	2010-01-01	True	0	0	1	87.84
4	2010-01-02	False	0	1	2	18.65

1. Find the mean, median, and mode for how many times visitors rode each ride. See Week 1 for Google advice on this.

In [26]:
```
dfnew = df.copy()
numeric_columns = dfnew.select_dtypes(include=['int64', 'float64']).columns.tolist(
mydf_numeric = dfnew[numeric_columns]

mean = mydf_numeric.mean()
median = mydf_numeric.median()
mode = mydf_numeric.mode()
```

2. Use groupby() to find the mean, median, and mode for how many times each ride was ridden on each given day.

Suggested Google search or ChatGPT prompt: I first tried: "How do I use groupby to find the mean over each day in my DataFrame?" but Google didn't help me.

Then I tried: "How do I find the mean over each date in a dataframe?" Sometimes, you have to try multiple searches.

This gives me a very helpful site, where someone is doing the same thing we are:

https://stackoverflow.com/questions/40788530/how-to-calculate-mean-of-some-rows-for-each-given-date-in-a-dataframe

In [27]:
```
# Group by the date column and calculate mean, median, and mode for each day
grouped_mean = dfnew.groupby('VisitDate')[numeric_columns].mean()
grouped_median = dfnew.groupby('VisitDate')[numeric_columns].median()
grouped_mode = dfnew.groupby('VisitDate')[numeric_columns].apply(lambda x: x.mode()

print("Mean by day:\n", grouped_mean, "\n")
print("Median by day:\n", grouped_median, "\n")
print("Mode by day:\n", grouped_mode, "\n")
```

```
Mean by day:
             MartianRide  TeacupRide  RiverRide  MoneySpent
VisitDate
2010-01-01         0.75        0.25       1.25     54.0625
2010-01-02         0.00        1.00       2.00     18.6500
2010-01-03         0.00        1.50       1.25     45.0850
2010-01-04         0.00        0.00       1.00     38.7100
2010-01-05         0.50        0.00       0.00      7.2800
...                 ...         ...        ...         ...
2023-12-27         0.50        0.50       1.00     44.0550
2023-12-28         6.00        0.00       0.00     58.5800
2023-12-29         0.00        1.00       3.00     54.8500
2023-12-30         3.00        0.00       1.00      9.5300
2023-12-31         0.00        0.00       1.00     40.0200

[4404 rows x 4 columns]

Median by day:
             MartianRide  TeacupRide  RiverRide  MoneySpent
VisitDate
2010-01-01          0.0         0.0        0.5      59.055
2010-01-02          0.0         1.0        2.0      18.650
2010-01-03          0.0         0.5        0.5      45.940
2010-01-04          0.0         0.0        1.0      38.710
2010-01-05          0.5         0.0        0.0       7.280
...                 ...         ...        ...         ...
2023-12-27          0.5         0.5        1.0      44.055
2023-12-28          6.0         0.0        0.0      58.580
2023-12-29          0.0         1.0        3.0      54.850
2023-12-30          3.0         0.0        1.0       9.530
2023-12-31          0.0         0.0        1.0      40.020

[4404 rows x 4 columns]

Mode by day:
 0           MartianRide  TeacupRide  RiverRide  MoneySpent
VisitDate
2010-01-01          0.0         0.0        0.0       10.30
2010-01-02          0.0         1.0        2.0       18.65
2010-01-03          0.0         0.0        0.0       32.79
2010-01-04          0.0         0.0        1.0       38.71
2010-01-05          0.0         0.0        0.0        0.77
...                 ...         ...        ...         ...
2023-12-27          0.0         0.0        0.0       39.03
2023-12-28          2.0         0.0        0.0       19.26
2023-12-29          0.0         1.0        3.0       54.85
2023-12-30          3.0         0.0        1.0        9.53
2023-12-31          0.0         0.0        1.0       40.02

[4404 rows x 4 columns]
```

3. Find the standard deviation and variance of the count for each ride.

Suggested Google search or ChatGPT prompt: "How do I find the standard deviation of a Series in pandas documentation?" and similar query for variance.

I included the word "documentation" because this task likely involves applying one single function for standard deviation and another for variance, so I expected that I could find a single documentation page for each that would cover my needs. I get these:

https://pandas.pydata.org/pandas-docs/dev/reference/api/pandas.Series.var.html

https://pandas.pydata.org/docs/reference/api/pandas.Series.std.html

In [28]:
```
# Find the standard deviation and variance for each ride
std_dev = mydf_numeric.std()
variance = mydf_numeric.var()

print(f"Standard Deviation:\n{std_dev}\n")
print(f"Variance:\n{variance}\n")
```

```
Standard Deviation:
MartianRide     2.077339
TeacupRide      1.232851
RiverRide       1.295757
MoneySpent     29.657410
dtype: float64

Variance:
MartianRide      4.315338
TeacupRide       1.519922
RiverRide        1.678987
MoneySpent     879.561951
dtype: float64
```

4. Find the 90th percentile count for each ride. That is, if the customers are ordered by their number of rides, and there are 100 customers, how many rides does the 90th person take? There is a function in pandas that easily does this calculation.

Suggested Google search or ChatGPT prompt: "How do I find a percentile value for a Series in pandas documentation?"

This time, I will let you find the page! Hint: it's not actually called the "percentile" function.

In [29]:
```
Ninety_percentile = mydf_numeric.drop(['MoneySpent'], axis=1).quantile(0.9)

print(f"Ride\t\t90th Percentile\n{Ninety_percentile}")
```

```
Ride            90th Percentile
MartianRide    2.0
TeacupRide     2.0
RiverRide      3.0
Name: 0.9, dtype: float64
```

5. Plot a histogram of the ride count, using each day as data element as you did in #2. Use both the total ride count for each day as well as, separately, the mean ride count for each day.

Suggested Google search or ChatGPT prompt: "How do I plot a histogram for a Series in pandas?"

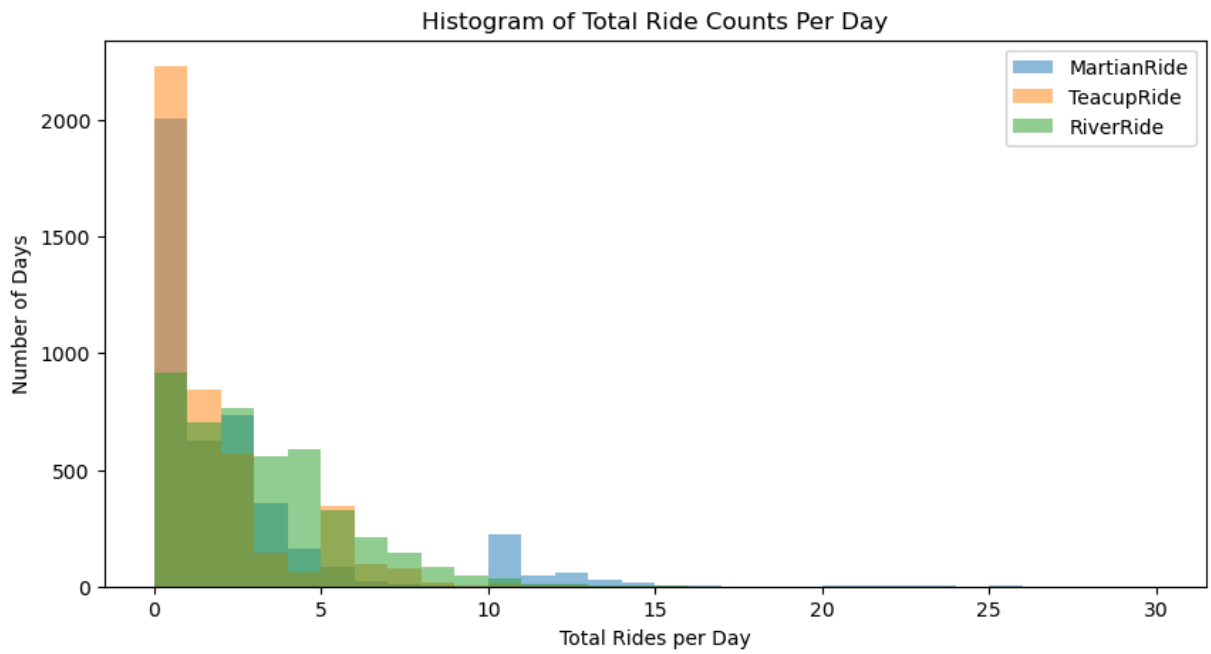
I found this: https://pandas.pydata.org/docs/reference/api/pandas.Series.plot.hist.html

Note: the "by" keyword will not help you plot a count for each day; if you try to use it for this, it will try to draw a separate histogram for each day.

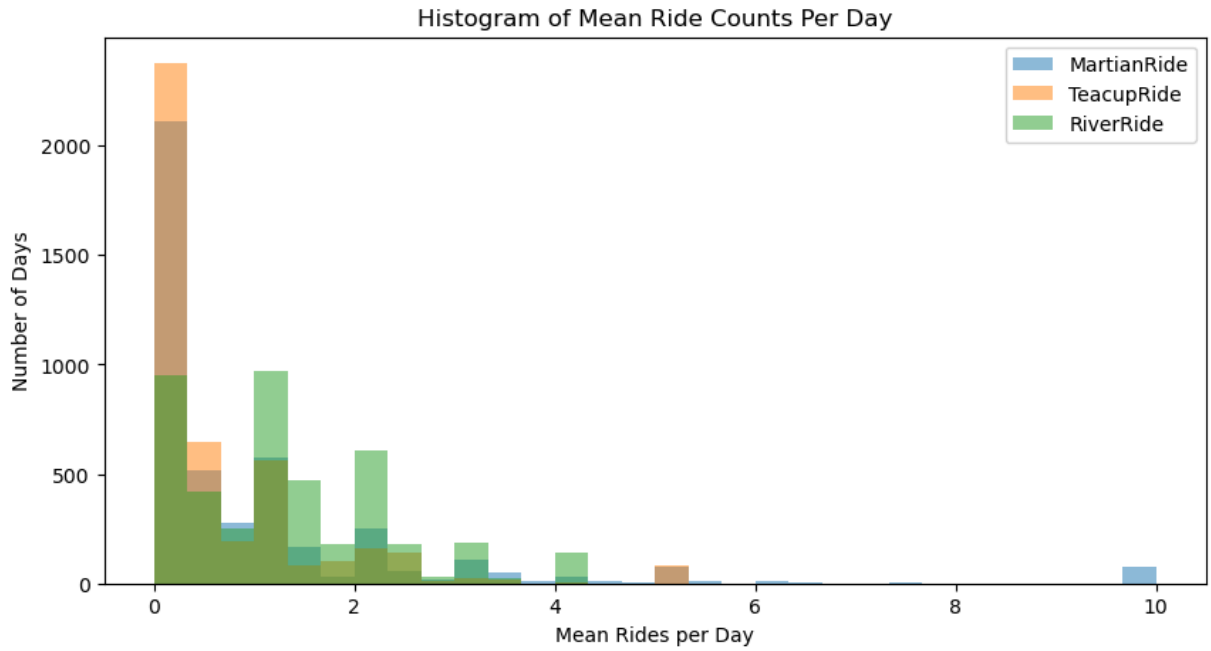
You've already learned how to find the mean ride count for each day. Can you use a similar idea to find the total ride count for each day?

Combined Histograms

In [30]:
```
rides_per_day = df.groupby("VisitDate")[["MartianRide", "TeacupRide", "RiverRide"]]
rides_per_day.plot(kind='hist', bins=30, alpha=0.5, figsize=(10,5), xlabel="Total R
```



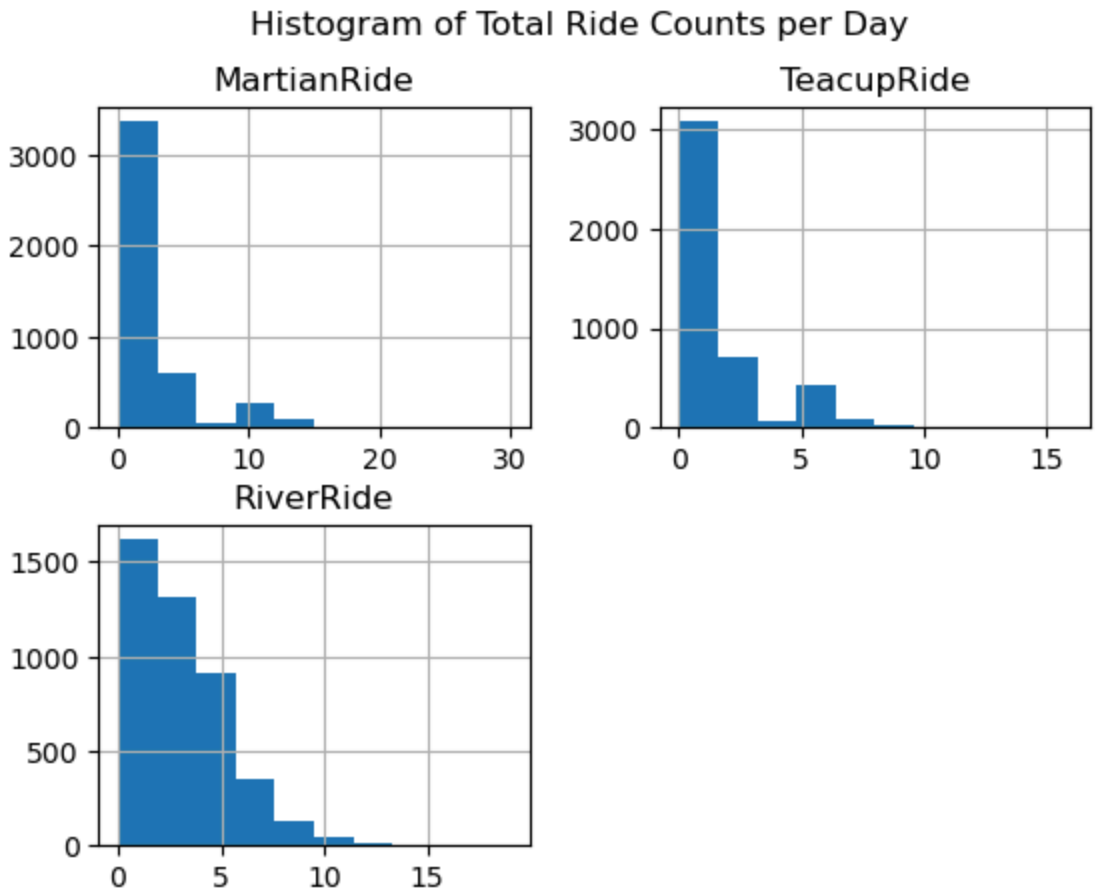
In [31]:
```
rides_per_day = df.groupby("VisitDate")[["MartianRide", "TeacupRide", "RiverRide"]]
rides_per_day.plot(kind='hist', bins=30, alpha=0.5, figsize=(10,5), xlabel="Mean Ri
```

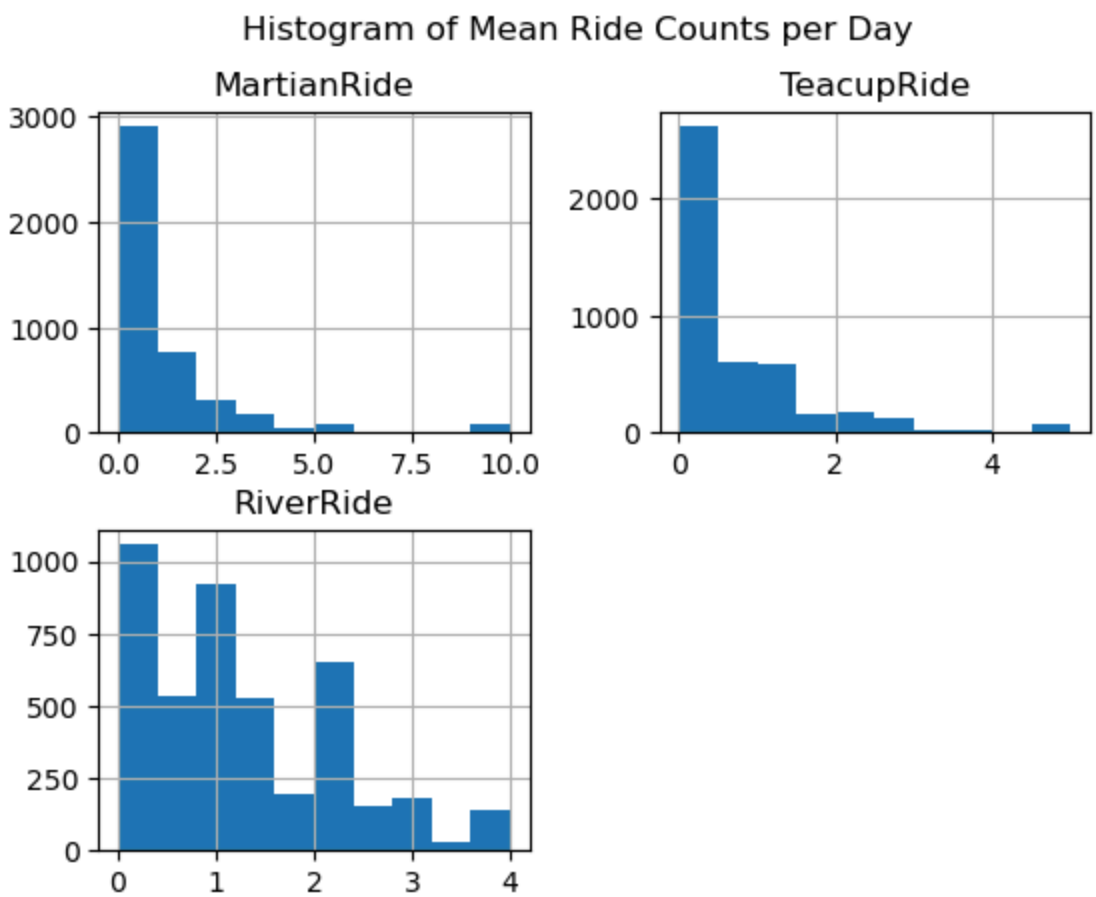


Separate Histograms

In [32]:
```
# drop the money spent column for grouping
dfnew_dropped = dfnew.drop(['MoneySpent'], axis=1)
numeric_columns = dfnew_dropped.select_dtypes(include=['int64', 'float64']).columns

# Calculate the total and mean ride counts for each day
grouped_sum = dfnew_dropped.groupby('VisitDate')[numeric_columns].sum()
grouped_mean = dfnew_dropped.groupby('VisitDate')[numeric_columns].mean()

# Histogram of total ride count for each day
_ = grouped_sum.hist()
plt.suptitle('Histogram of Total Ride Counts per Day')

# Histogram of mean ride count for each day
_ = grouped_mean.hist()
plt.suptitle('Histogram of Mean Ride Counts per Day')
```

Out[32]:
```
Text(0.5, 0.98, 'Histogram of Mean Ride Counts per Day')
```


Histogram of Total Ride Counts per Day
MartianRide
TeacupRide
RiverRide


Histogram of Mean Ride Counts per Day
MartianRide
TeacupRide
RiverRide

6. Make a bar chart showing the total number of Adult and Child participants.

Try to formulate your own Google search or ChatGPT prompt.

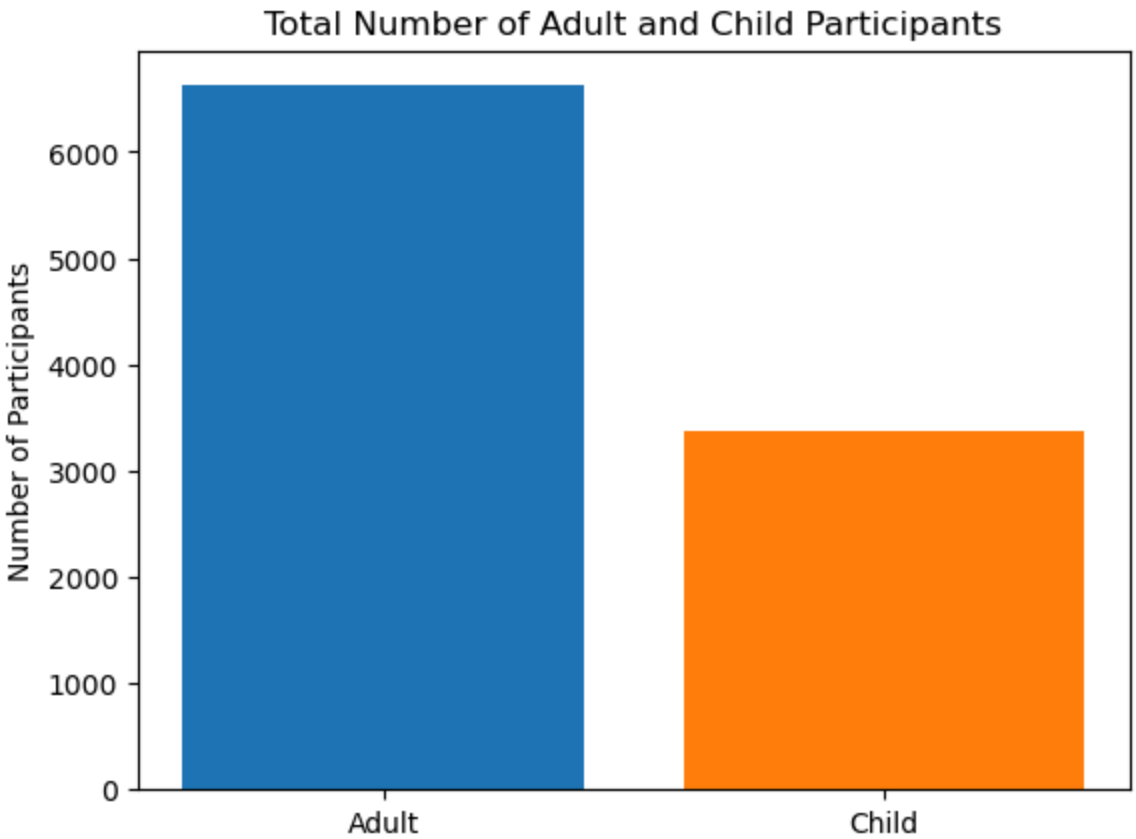
Here's one page I found:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html

which shows some interesting examples for you to use. I recommend clicking on the first, basic example to see how to use the function.

You can also look up the pandas documentation, which might lead to an entirely different method.

In [33]:
```
# Count the number of Adult and Child participants
adult_count = df[df['IsAdult'] == True].shape[0]
child_count = df[df['IsAdult'] == False].shape[0]

# Create the bar chart
categories = ['Adult', 'Child']
counts = [adult_count, child_count]
bar_colors = ['tab:blue', 'tab:orange']

fig, ax = plt.subplots()
ax.bar(categories, counts, color=bar_colors)

ax.set_ylabel('Number of Participants')
ax.set_title('Total Number of Adult and Child Participants')

plt.show()
```



7. Make a stacked bar chart showing how many Adult and Child visits took the River Ride, with the x-axis showing the number of rides and with two stacked bars (adult, child). That is, there could be a column for the number of Adults (and Children) who took 0 rides, 1 ride, 2 rides, and so on.

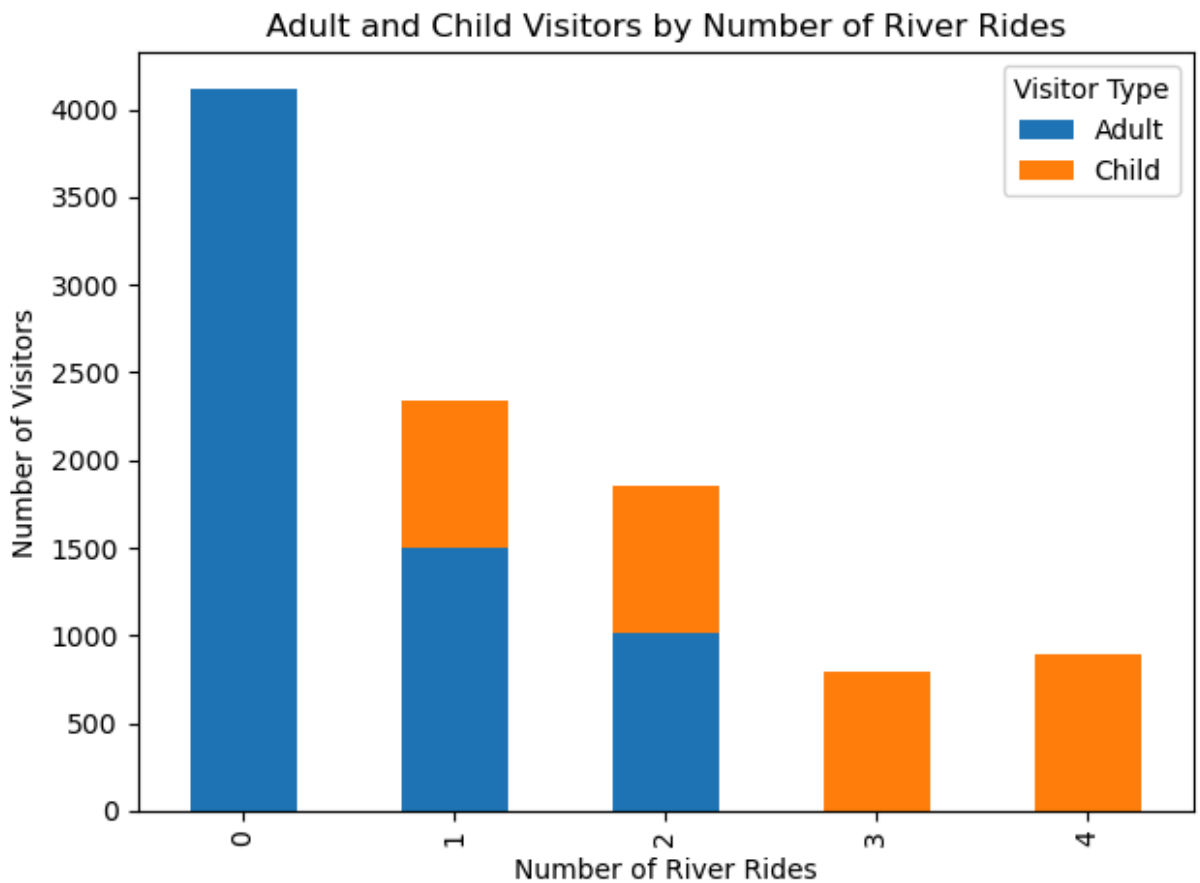
Try to formulate your own Google search or ChatGPT prompt.

For me, the pandas documentation was easier than the matplotlib documentation.

- If you use the pandas method, you might have to one-hot encode the IsAdult column. You might want names for the new columns other than True and False. Use df.join or pd.concat to attach the pd.get_dummies table you've created, and then groupby the RiverRide value.
- You will need to end up with a small table with two columns (IsAdult True and False) and an index (RiverRide) as well as a small number of rows (the values of RiverRide)
- That said, if you can come up with another approach, it's fine.

In [34]:
```
# Create one-hot encoded columns for IsAdult
df_encoded = pd.get_dummies(df['IsAdult'], prefix='')
df_encoded.columns = ['Child', 'Adult']

# Combine with RiverRide column
df_with_encoded = pd.concat([df[['RiverRide']], df_encoded], axis=1)
```

```
# Group by RiverRide and sum to get counts
stacked_data = df_with_encoded.groupby('RiverRide')[['Adult', 'Child']].sum()

# Create stacked bar chart
ax = stacked_data.plot(kind='bar', stacked=True, color=['tab:blue', 'tab:orange'])
ax.set_xlabel('Number of River Rides')
ax.set_ylabel('Number of Visitors')
ax.set_title('Adult and Child Visitors by Number of River Rides')
plt.legend(title='Visitor Type')
plt.tight_layout()
plt.show()
```



8. Other tasks

Use seaborn (import seaborn as sns) to create a kernel density estimation (kde) plot. (You can choose any aspect of the amusement park dataset to plot.) Here is a tutorial you can read about seaborn:

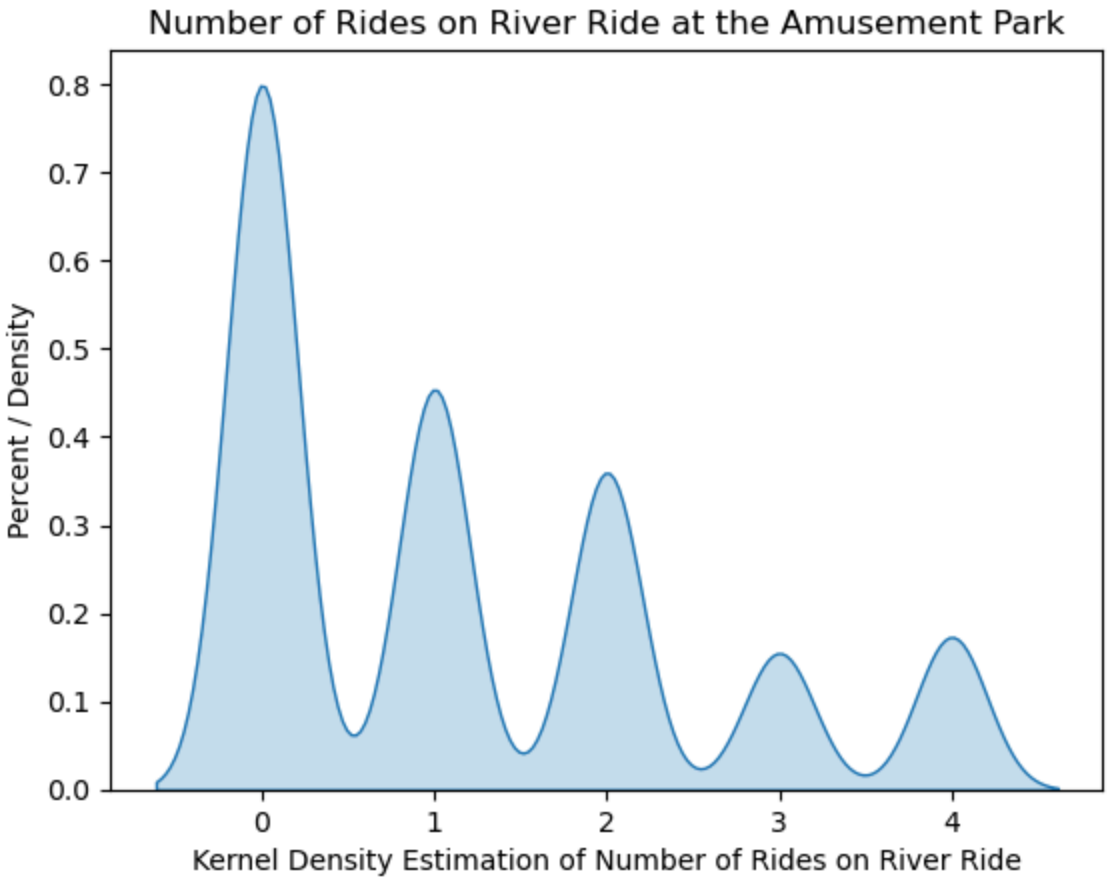
https://seaborn.pydata.org/tutorial/introduction.html

Seaborn often makes nicer looking graphs than pandas or matplotlib.

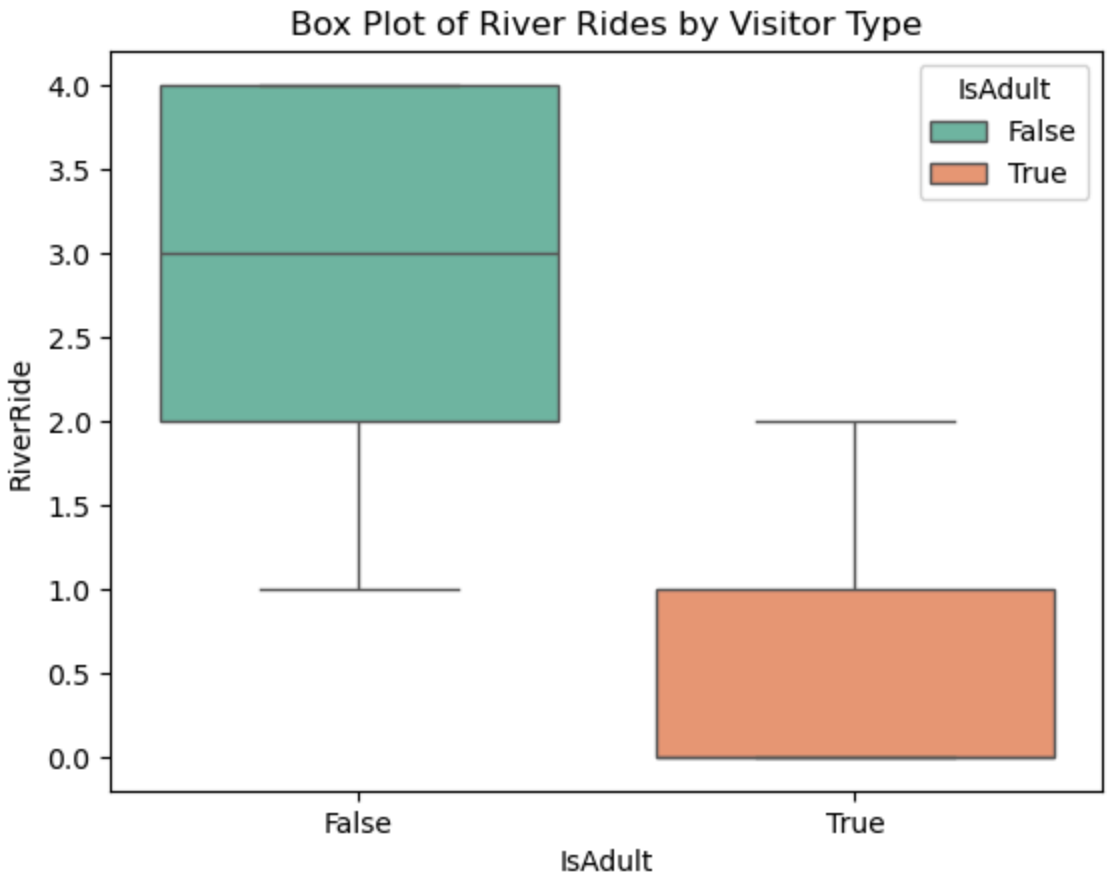
Now use seaborn to make a swarm plot, a violin plot, and a box plot.

Which plots are the best for showing this data?

In [35]:
```
# kernel density estimation (KDE) plot using seaborn

sns.kdeplot(data=df, x='RiverRide', fill=True)
plt.xlabel('Kernel Density Estimation of Number of Rides on River Ride')
plt.ylabel('Percent / Density')
plt.title('Number of Rides on River Ride at the Amusement Park')
plt.show()
```



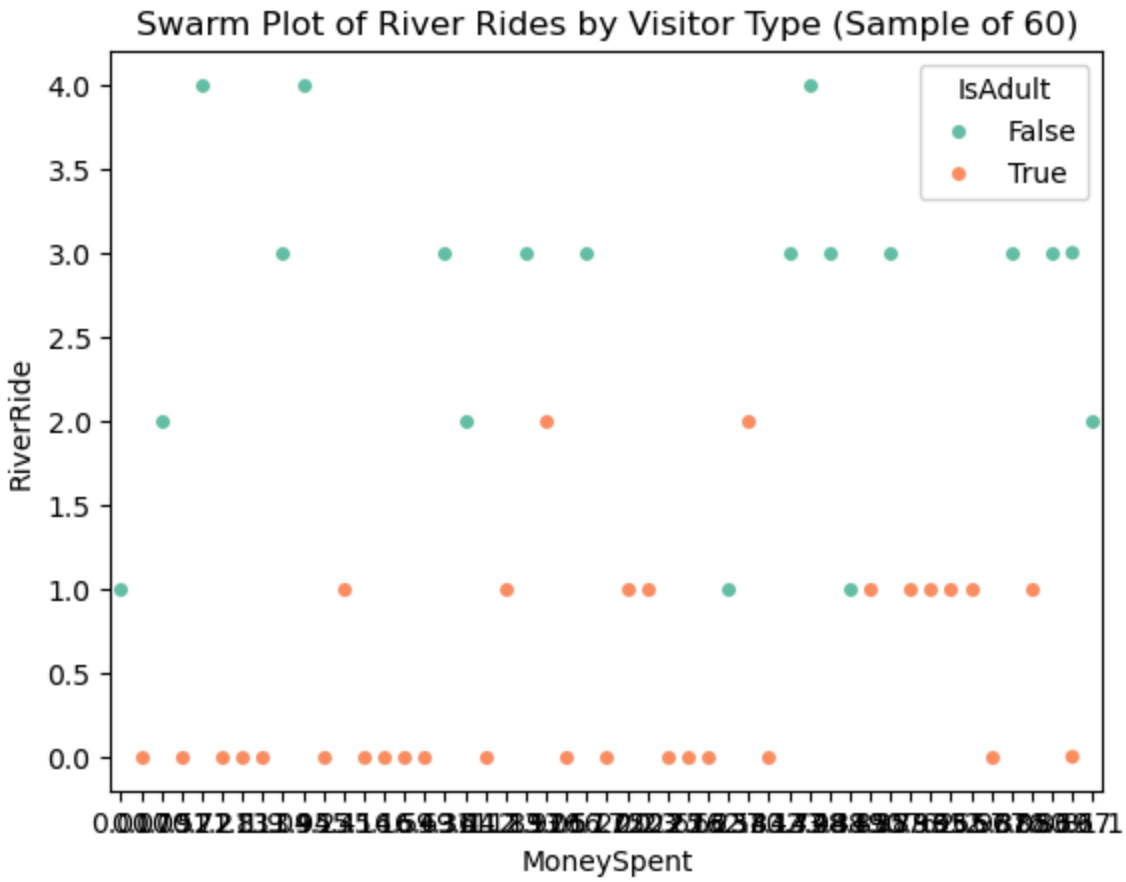
In [36]:
```
boxplot = sns.boxplot(x='IsAdult', y='RiverRide', data=df, palette='Set2', hue='IsA
boxplot.set_title('Box Plot of River Rides by Visitor Type')
plt.show()
```



In [37]:
```
violin_plot = sns.violinplot(x='IsAdult', y='RiverRide', data=df, palette='Set2', h
violin_plot.set_title('Violin Plot of River Rides by Visitor Type')
plt.show()
```

Looking at these plots I like the Violin plot the best

In [38]:
```
# Swarm plots are computationally expensive with large datasets
# Sample a smaller subset for visualization
df_sample = df.sample(n=50, random_state=42)
swarmplot = sns.swarmplot(x='MoneySpent', y='RiverRide', data=df_sample, palette='S
swarmplot.set_title('Swarm Plot of River Rides by Visitor Type (Sample of 60)')
plt.show()
```



4. Storytelling With Data graph

Try to make a scatterplot using amusement park dataset that is similar to the one on page 45, where the two axes are the Teacup Ride count and the River Ride count. If you want to see individual dots and not a dense swarm of dots, you'll have to cut the dataset down to a small number of points (say, 100 points or so). You can remove the other points, for instance, and focus on the first 100 points.

Here are some things you could do (you don't have to do all of them):

- Draw a dashed line that roughly separates the adult from child points.
- Draw the points in the adult vs. child region of the graph in different colors.
- Choose the right size and number of dots to make the graph look good.
- Write a word that appears on the dashed line (like AVG in the plot on page 45).

If there are any other graphs in the Storytelling With Data chapter that look interesting, and you want more practice, you can try to reproduce them too.

Display the various field in the df

In [39]:
```
df.head(10)
```

Out[39]:

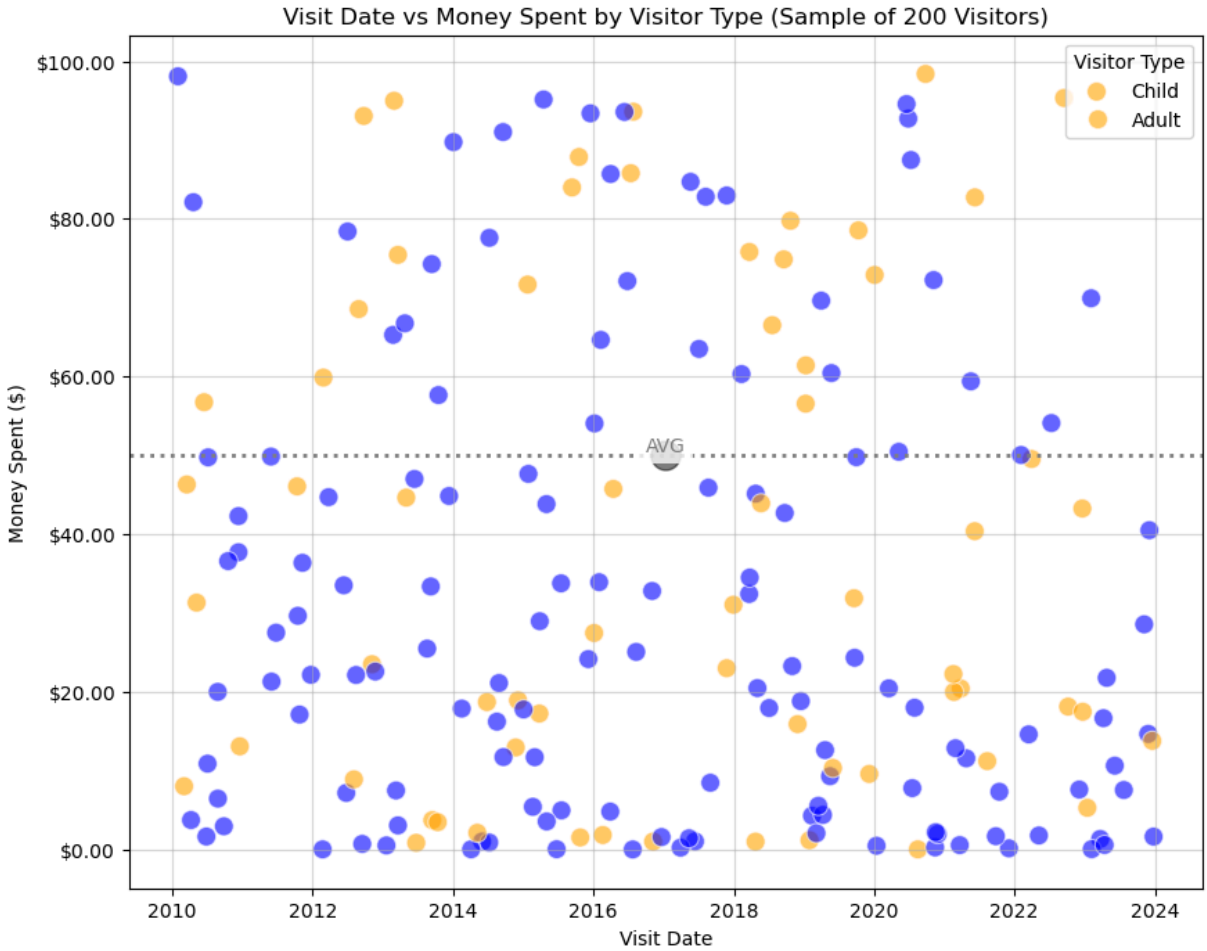
	VisitDate	IsAdult	MartianRide	TeacupRide	RiverRide	MoneySpent
0	2010-01-01	False	0	1	4	10.30
1	2010-01-01	True	3	0	0	38.77
2	2010-01-01	True	0	0	0	79.34
3	2010-01-01	True	0	0	1	87.84
4	2010-01-02	False	0	1	2	18.65
5	2010-01-03	False	0	1	4	47.17
6	2010-01-03	True	0	0	0	44.71
7	2010-01-03	True	0	0	0	32.79
8	2010-01-03	False	0	5	1	55.67
9	2010-01-04	True	0	0	1	38.71

Scatterplot of amusement park data

sample first 200 points to avoid dense swarm

In [40]:
```
from matplotlib.ticker import StrMethodFormatter
df_scatter = df.sample(200)
plt.figure(figsize=(10, 8))

# Create scatterplot with different colors for Adults vs Children
sns.scatterplot(data=df_scatter, x='VisitDate', y='MoneySpent', hue='IsAdult',
                palette={True: 'blue', False: 'orange'}, s=100, alpha=0.6)
ax = plt.gca()
ax.yaxis.set_major_formatter(StrMethodFormatter('${x:,.2f}'))

#
# Dotted horizontal line at y=50 with a label in the middle
#
y_line = 50
xmin, xmax = plt.xlim()
#
# Draw the horizontal line
#
plt.hlines(y_line, xmin, xmax, colors='gray', linestyles=':', linewidth=2)
plt.text(((xmin + xmax) / 2), y_line, 'AVG',
         ha='center', va='bottom', color='gray',
         bbox=dict(facecolor='white', edgecolor='none', alpha=0.8))
plt.xlim(xmin, xmax)
#
# Mark the midpoint of the line with a gray circle
#
x_mid = (xmin + xmax) / 2
```

```
plt.plot(x_mid, y_line, marker='o', color='black', markersize=15, alpha=0.5)
#
#
plt.legend(title='Visitor Type', labels=['Child', 'Adult'])
plt.xlabel('Visit Date')
plt.ylabel('Money Spent ($)')
plt.title('Visit Date vs Money Spent by Visitor Type (Sample of 200 Visitors)')
plt.grid(True, alpha=0.5)

plt.show()
```



Additional helpers

I used the view data built-in tool to check the validity of the dataset several times. It was easy to sort a column ascending and then descending to quickly inspect it for anomalies.

I needed to consult the view chat option in VS Code in order to get some context help to debug my code to make the annotations on the chart.